ACTIA®
ENGINEERING SERVICES

Embedded Team Academy

Advanced C++ training

Prepared and Presented by
Mahdi SAKKA

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

Agenda

Module 1: C++ Basics

Module 2: C++ Object Oriented Programming

Module 3: C++ Standard Library

Module 4: C++ Advanced Programming

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

Agenda

Module 2: C++ Object Oriented Programming

- Introduction to OOP
- Classes and structures
- Special methods
- Access specifiers
- Static and non-static member functions
- Friend functions and classes
- Inheritance
- Polymorphism
- Overloading and overriding
- Encapsulation and abstraction
- Operators overloading
- Objects relationships
- Quiz

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

Introduction to OOP

- Procedural programming:
  - List of instructions into functions combined together
  - Data and functions are separate entities
  - Manage combination of both of them to get desired result
- What leads us to use OOP ?
  - Everything around us are objects: tables, seats, doors, food, cars, PCs, ...
  - Objects → Properties (attributes)
    → Behavior (methods)

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

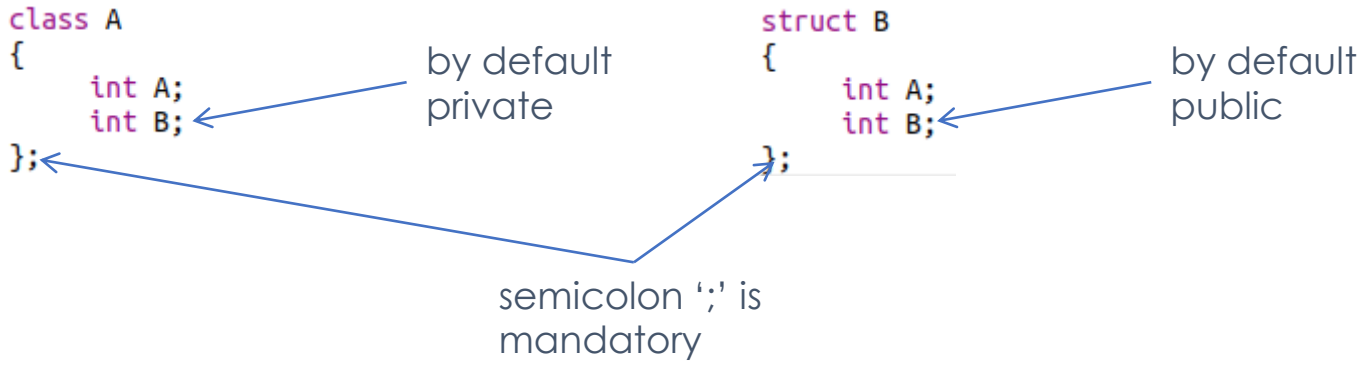
Introduction to OOP

- Object Oriented Programming advantages
  - Easier way for code maintenance / enhancement (modular code)
  - Tie together data and behavior
  - Intuitive way to work with data



Classes and structures

- Class is a pattern that represent object (may create many instances of same class)
- Struct is same as class but …



- Classes (structs) holds data and functions to manipulate data

Classes and structures

- Class member functions (used to manipulate data)

employee.h

```
class Employee
{
public:
    std::string m_name {""};
    int m_id {0};
    double m_wage {0.0};

    // Print employee information to the screen
    void print()
    {
        std::cout << "Name: " << m_name <<
                "  Id: " << m_id <<
                "  Wage: $" << m_wage << std::endl;
    }
};
```

employee.cpp

```
void Employee::print()
{     std::cout << "Name: " << m_name <<
            "  Id: " << m_id <<
            "  Wage: $" << m_wage << std::endl;
}
```

```
int main()
{
    // Declare two employees
    Employee alex { "Alex", 1, 25.00 };
    Employee joe { "Joe", 2, 22.25 };

    // Print out the employee information
    alex.print();
    joe.print();

    return 0;
}
```

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

ACTIA®

Classes and structures

Class member functions

```
void x()
{
// You can't call y() from here unless the compiler has already
//seen a forward declaration for y()
}

void y()
{
}
```

```
class Test
{
public:
     void x() { y(); } // okay to call y() here, even though y()
                       // isn't defined until later in this class
     void y() { };
};
```

Class members (methods / attributes) can use each other since they belong all to class scope

Define nested types and alias that belong to class

```
struct Employee
{
    enum Contract // nested type
    {
        permanent,
        temporary,
        contractor,
    };
    using IDType = int; // nested type alias

    IDType id{};
    Contract contract{};
};
```

Access specifiers

- C++ classes has 3 access specifiers:
  - «public»: visible from the inside and outside of the class
  - «protected»: visible from inside and child classes
  - «private»: visible only inside the class

```
class Employee
{
public:
    // Print employee information to the screen
    void print();
protected:
    int m_id {0};
private:
    std::string m_name {""};
    double m_wage {0.0};
};
```

```
int main()
{
    //Employee alex { "Alex", 1, 25.00 }; // error
    Employee alex; // Ok

    // Print out the employee information
    alex.print(); // Ok
    alex.m_id = 2; // error not visible
    alex.m_name = "Joe"; // error not visisble

    return 0;
}
```

Special methods

- C++ class define many special methods types:
  - Constructor
  - Destructor
  - Copy constructor
  - Move constructor

Special methods

- C++ class define many special methods types:
  - Constructor
  - Destructor
  - Copy constructor
  - Move constructor
- Adding keywords «default» or «delete» in front of constructors (optionally) will either use default constructor implementation or delete that constructor

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

ACTIA®

Special methods

- **Constructor**: special method called automatically when object is instantiated
  - Must have same name as class name
  - Typically used to initialize member variables of the class to appropriate user-provided values, or to do any setup steps necessary for the class to be used
  - Two types of constructors:
    - Default
    - User defined

☐ Provisoire ☐ Public ☐ Interne ☒ Confidentiel ☐ Secret

ACTIA®

Special methods

Constructor

Default constructor

«default» key word to indicate to compiler to use default implementation

User defined constructor

attributes initialization

```
class Employee
{
public:
    enum class Contract // nested type
    {
        permanent,
        temporary,
        contractor,
    };
    using IDType = int; // nested type alias
    Employee() = default;
    Employee(IDType id, Contract contract, const std::string& name);
private:
    IDType id{};
    Contract contract{};
    std::string m_name {""};
};
```

```
Employee::Employee(IDType id,
                   Contract contract,
                   const std::string& name) : id(id),
                                              contract(contract),
                                              m_name(name)
{
    // more initialization if needed
}
```

- If no constructor is defined, default constructor is created implictly

ACTIA®

Special methods

- **Constructor:** constructor initialization list

This is not an initialization, but assignment



```
class Test
{
public:
     Test(const std::string& str, int i = 1);
private:
     int var1;
     std::string var2;
     bool var3;
}

Test::Test(const std::string& str, int i)
{
     var1 = i;
     var2 = str;
     var3 = false;
}
```

Special methods

- **Constructor:** constructor initialization list

  * Default parameter value, should be always at the end of parameters
  * Can have many default parameters, but should be always kept at the end

  Initialization list: initalise attributes ',' separated

```
class Test
{
public:
     Test(const std::string& str, int i = 1);
private:
     int var1;
     std::string var2;
     bool var3;
};

Test::Test(const std::string& str, int i) : var1{i},
                                            var2{str},
                                            var3{true}
{
     // initialization list will be called before
     // processing here
}


int main()
{
     Test t1("Hello", 10);
     Test t2("Default decond parameter");

    return 0;
}
```

- Make sure to keep order of declaration within the initialization list

Special methods

- **Constructor:** call of constructor

```
int main()
{
     // call of the default constructor
     Employee alex;

     // call of the constructor corresponding to provided
     // parameters in right order
    Employee joe{1, Employee::Contract::permanent, "Joe"};

    return 0;
}
```

Default constructor
(without parameters)

Call of corresponding
defined constructor

- May have as many constructors as needed (with different signature)
- If a custom constructor is defined, default constructor can no longer be called unless defined

☐ Provisoire ☐ Public ☐ Interne ☒ Confidentiel ☐ Secret

ACTIA®

Special methods

Constructor: call of constructor

Class D has composition of both classes B and C

```
class B
{
public:
    B(int x) { std::cout << "B constructor " << x << std::endl; }
};
class C
{
public:
    C() { std::cout << "C default constructor" << std::endl;  }
};
class D
{
public:
     D() : b(1) { std::cout << "D constructor" << std::endl; }
private:
     C c;
     B b;
};
int main()
{
     D d;
     return 0;
}
```

Execution result

```
C default constructor
B constructor 1
D constructor
```

Special methods

- **Constructor:** call of constructor
  - Different ways to call constructor

```
class Test
{
public:
     Test(const std::string& str, int i = 1);
private:
     int var1;
     std::string var2;
     bool var3;
};

Test::Test(const std::string& str, int i)
{
     var1 = i;
     var2 = str;
     var3 = false;
}


int main()
{
     Test t1(std::string("Hello"));
     Test t2 = Test("Hello");
     Test t3 = std::string("Hello");

    return 0;
}
```

Special methods

- **Copy Constructor:** called during instantiation by copy or assigning an object to another
- Declaration:

```
class_name(const class_name&
rhs);
```

- If **not** defined, default copy constructor will be created implictly (automatically copy all members values from class being copied (if no other constructors are defined).
- If defined, initialization of members has to be done in the copy constructor definition

Special methods

Copy Constructor:

The copy constructor is not called here, compiler elides (deletes) the call of copy constructor

The copy constructor is called here

```
class Something
{
public:
    Something() = default;
    Something(const Something&)
    {
        std::cout << "Copy constructor called\n";
    }
};


int main()
{
    std::cout << "Initializing s1\n";
    Something s1 = Something(); // copy constructor not called here

    std::cout << "Initializing s2\n";
    Something s2 = s1; // copy constructor called here
}
```

- Can prevent copying an object by deleting copy constructor or declaring it as private

Special methods

- **Move Constructor:** move ownership of the resources from one object to another (which is typically much less expensive than making a copy).
- Declaration:

```
class_name(class_name&& rhs);
```

- If **not** defined, default move constructor will be created implictly (automatically copy all members values from class being copied (if no other constructors are defined).
- If defined, initialization of members has to be done in the move constructor definition

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

ACTIA®

Special methods

Move Constructor:

The default constructor is called here

The move constructor is called here

```
class Something
{
public:
    Something() = default;
    Something(Something&&)
    {
        std::cout << "Move constructor called\n";
    }
private:
    std::string value{"Hello"};
};
int main()
{
    std::cout << "Initializing s1\n";
    Something s1 = Something{};

    std::cout << "Initializing s2\n";
    Something s2 = std::move(s1);
}
```

- Can prevent moving an object by deleting move constructor or declaring it as private

Special methods

Move semantic: std::move is a standard library function that casts (using static_cast) its argument into an r-value reference, so that move semantics can be invoked.

3 copies (expensive in time and memory)

```
void swapCopy(std::string& a, std::string& b)
{
     std::string tmp { a }; // invokes copy constructor
     a = b; // invokes copy assignment
     b = tmp; // invokes copy assignment
}
int main()
{
     std::string x{ "abc" };
     std::string y{ "de" };

     std::cout << "x: " << x << "# y: " << y << std::endl;
     swapCopy(x, y);
     std::cout << "x: " << x << "# y: " << y << std::endl;
     return 0;
}
```



3 moves(**less** expensive in time and memory)

```
void swapMove(std::string& a, std::string& b)
{
     std::string tmp { std::move(a) }; // invokes move constructor
     a = std::move(b); // invokes move assignment
     b = std::move(tmp); // invokes move assignment
}
int main()
{
     std::string x{ "abc" };
     std::string y{ "de" };

     std::cout << "x: " << x << "# y: " << y << std::endl;
     swapMove(x, y);
     std::cout << "x: " << x << "# y: " << y << std::endl;
     return 0;
}
```

Special methods

- **Move semantic:** moved object are no longer valid (or in undefined state)
- To avoid any issues: Moved objects shall be set to default / zero state and shall not be used

```
std::string s1{"Test"};
std::string s2{std::move(s1)};
// move well managed s1 is invalid after move
std::cout << "Move string " << s2 << " old string " << s1 << std::endl;
```

- The value of moved-from objects will have an undefined value (may keep its value or have another arbitrary value)

Special methods

- **Destructor**: special method called just before object is destructed
- Destructor should be always virtual (will be explained later)
- Should never throw exceptions

virtual ~class_name();

```
class Simple
{
public:
    Simple()
    {
        std::cout << "Constructing Simple " << '\n';
    }
    virtual ~Simple()
    {
        std::cout << "Destructing Simple" << '\n';
    }
};

int main()
{
    // Allocate a Simple on the stack
    Simple simple;
    // Allocate a Simple dynamically
    Simple* pSimple{ new Simple };
    // We allocated pSimple dynamically, so we have to delete it.
    delete pSimple;
    return 0;
} // simple goes out of scope here
```

Destructor for pointer is called here

Destructor for object is called here

Special methods

- **RAII (Resource Aquisition Is Initialization)**
  - It is a C++ programming technique to
    - Guarantees that the resource is available to any function that may access the object
    - Guarantees that all resources are released when the lifetime of their controlling object ends, in reverse order of acquisition
  - It means that the object resource is allocated in the constructor and freed in the destructor
  - Then, the resource can be used when object is alive

Special methods

- Current object pointer «**this**»
  - It is pointing to the object being operated on
  - Each object has its own «this» pointer

```
class Something
{
private:
    int data;

public:
    Something(int data)
    {
        this->data = data; // this->data is the member, data is the local parameter
    }
};
```

Special methods

- Current object pointer «**this**» : chaining member functions
  - Return a reference to current object instance
  - Use it for another call with object instance updated

```
int main()
{
    Calc calc{};
    calc.add(5).sub(3).mult(4);

    std::cout << calc.getValue() << '\n';
    return 0;
}
```



```
class Calc
{
private:
    int m_value { 0 };
public:
    Calc& add(int value)
    {
        m_value += value;
        return *this;
    }
    Calc& sub(int value)
    {
        m_value -= value;
        return *this;
    }
    Calc& mult(int value)
    {
        m_value *= value;
        return *this;
    }

    int getValue()
    {
        return m_value;
    }
};
```

Special methods

Implicit and explicit constructor call

```
class Test
{
public:
     Test(int x, int y = 1): m_x{x}, m_y{y}
     { }
     explicit Test(const std::string& str) : m_str{str}
     { }

     int getX() const
     {
          return m_x;
     }
     std::string getStr() const
     {
          return m_str;
     }

private:
     int m_x;
     int m_y;
     std::string m_str;
};
```



```
void printTest(const Test& t)
{
     std::cout << t.getX();
}
void printStr(const Test& t)
{
     std::cout << t.getStr();
}

int main()
{
     printTest(6);
     // copile error (no implicit conversion)
     printStr(std::string("Test"));

     return 0;
}
```

Access specifiers

- C++ possible access specifiers: public, protected and private
- Any class member function / attribute that is:
  - public: can be used inside the class and from calling objects
  - protected: can be used inside the class and from child classes if any
  - private: can be used only inside the class

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

ACTIA®

Exercice

- Create a class named «Object» that allows
  - Creating all constructors
  - Containing attributes: m_name(string), m_id(int), m_pNext (Object*)
  - Initializes the attributes and suitable processing in different constructors

☐ Provisoire ☐ Public ☐ Interne ☒ Confidentiel ☐ Secret

ACTIA®

Static and non-static member functions

- Static member variables: shared by all instances
- Do not depend on class instances

```
class Something
{
public:
    int s_value{ 1 };
};


int main()
{
    Something first;
    Something second;

    first.s_value = 2;

    std::cout << first.s_value << '\n';
    std::cout << second.s_value << '\n';
    return 0;
}
```

→ 2
1

```
class Something
{
public:
    static int s_value;
};

int Something::s_value{ 1 };

int main()
{
    Something first;
    Something second;

    first.s_value = 2;

    std::cout << first.s_value << '\n';
    std::cout << second.s_value << '\n';
    return 0;
}
```

→ 2
2

Static and non-static member functions

- Static member variables initialization
  - Outside the class declaration(even if it is declared as private)

```
int Something::s_value{ 1 }; // defines the static member variable
```

  - Inside the class declaration, if «**static const**» variable declaration (inline declaration)

```
public:
    static const int s_value{ 4 }; // a static const int can be declared and initialized directly
```

- Call of static member variable

```
Something::s_value = 2; // call using scope resolution operator ::
instance.s_value = 2; // call using member access operator
```

- Must be declared outside the class if to be used in-there

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

ACTIA®

Static and non-static member functions

- Static member functions: like static member variables, they are not attached to instances
- Shared by all instances
- Do not need instantiation to be call

```
class Something
{
private:
    static int s_value;
public:
    static int getValueStatic() { return s_value; } // static member function
    int getValue() const { return s_value; }
};

int Something::s_value{ 1 }; // initializer

int main()
{
    std::cout << Something::getValueStatic() << '\n';
    Something object;
    std::cout << object.getValue() << '\n';
}
```

No need for instance to be called

Possible to call from instance since it is shared by all instances

Static and non-static member functions

- Static member functions **rule**: «this» pointer cannot be called from static member function
- Example of use

```
class IDGenerator
{
private:
    static int s_nextID; // Here's the declaration for a static member
public:
     static int getNextID(); // Here's the declaration for a static function
};

// Here's the definition of the static member outside the class.  Note we don't use the static keyword here.
// We'll start generating IDs at 1
int IDGenerator::s_nextID{ 1 };

// Here's the definition of the static function outside of the class.  Note we don't use the static keyword here.
int IDGenerator::getNextID() { return s_nextID++; }

int main()
{
    for (int count{ 0 }; count < 5; ++count)
        std::cout << "The next ID is: " << IDGenerator::getNextID() << '\n';

    return 0;
}
```

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

ACTIA®

Friend functions and classes

- Friend function is a function that can access even private members
- It can be either a normal function or another class member function

```
class A
{
private:
    int m_value { 0 };
public:
    void add(int value) { m_value += value; }
    // Make the reset() function a friend of this class
    friend void reset(A& a);
};
void reset(A& a)
{
    a.m_value = 0;// And can access the private data
}
int main()
{
    A a;
    a.add(5); // add 5
    reset(a); // reset to 0
    return 0;
}
```

```
class B;
class A
{
private:
    int m_value { 0 };
public:
    void add(int value) { m_value += value; }
    // Make the reset() function a friend of this class
    friend void B::reset(A& a);
};
void B::reset(A& a)
{
    a.m_value = 0;// And can access the private data
}
int main()
{
    A a; B b;
    a.add(5); // add 5
    b.reset(a); // reset to 0
    return 0;
```

Friend functions and classes

- Multiple friends:
  - A function or a member function can be a friend to multiple classes

```
class B;
class A
{
public:
    // Make the reset() function a friend of this class
    friend void reset(A& a, B& b);
};
class B
{
public:
    // Make the reset() function a friend of this class
    friend void reset(A& a, B& b);
};
void reset(A& a, B& b)
{
     // can access A and B private members
}
int main()
{
    A a;
    B b;
    reset(a, b);
    return 0;
}
```

Friend functions and classes

- Friend classes: same friend concept, but a friend class can access any private member at any part of the class

```
class B;
class A
{
private:
     int m_value;
public:
    // Make class B a friend of this class
    friend class B;
};
class B
{
public:
    // A privates can be accessed from B class
    void useA(A& a) { a.m_value = 0; };
};
int main()
{
    A a;
    B b;
    b.useA(a);
    return 0;
}
```

Inheritance

- Inheritance represents the relationship «is-a»
- An object inherits from another one means that it take the attributes and behavior of the parent class (of course while respecting the access rules)
- We call «parent» or «base» the class that will be inherited
- We call «child» or «derived» the class that will inherit from «base»
- Inheritance declaration

```
class ChildName : access_specifier ParentClassName
```

- Inheritance allows regrouping common behavior and data in base class (avoid duplication)

Inheritance

- Inheritance access specifier: **public**

Access specifier in base class	Access specifier when public inheritance
Public	Public
Protected	Protected
Private	Inaccessible

```
class Base
{
public:
    int m_public {};
protected:
    int m_protected {};
private:
    int m_private {};
};
class Pub: public Base // note: public inheritance
{
public:
    Pub()
    {
        // okay: m_public was inherited as public
        m_public = 1;
        // okay: m_protected was inherited as protected
        m_protected = 2;
        // not okay: m_private is inaccessible from derived
        //m_private = 3;
    }
};
```

- Always use public inheritance unless other type needed

Inheritance

- Inheritance access specifier: **protected** inheritance

Access specifier in base class	Access specifier when protected inheritance
Public	Protected
Protected	Protected
Private	Inaccessible

- Its usage is very rare

```
class Base
{
public:
    int m_public {};
protected:
    int m_protected {};
private:
    int m_private {};
};
class Prot: protected Base // note: public inheritance
{
public:
    Prot()
    {
        // okay: m_public was inherited as protected
        m_public = 1;
        // okay: m_protected was inherited as protected
        m_protected = 2;
        // not okay: m_private is inaccessible from derived class
        //m_private = 3;
    }
};
int main()
{
    Prot prot;
    //error: 'int Base::m_public' is inaccessible within this context
    prot.m_public = 0;
    return 0;
}
```

Inheritance

- Inheritance access specifier: **private** inheritance

Access specifier in base class	Access specifier when private inheritance
Public	Private
Protected	Private
Private	Inaccessible

- Next hierarchy won't see any attribute of base class

```
class Base
{
public:
    int m_public {};
protected:
    int m_protected {};
private:
    int m_private {};
};
class Priv: private Base // note: public inheritance
{
public:
    Priv()
    {
        // Priv: m_public becomes private
        m_public = 1;
        // okay: m_protected becomes private
        m_protected = 2;
        // not okay: m_private is inaccessible from derived class
        //m_private = 3;
    }
};
int main()
{
    Prot prot;
    //error: 'int Base::m_public' is inaccessible within this context
    prot.m_public = 0;
    return 0;
}
```

Inheritance

- Order of construction of derived class
  - In case of chain of inheritance, the first base class constructor is called first and then derived constructors are called in the order and last derived class constructor is called at the end

Base class constructor is called first

Derived class constructor is called at the end

```
class Base
{
public:
    int m_id {};

    Base(int id=0)
        : m_id { id }
    {
    }
};

class Derived: public Base
{
public:
    double m_cost {};

    Derived(double cost=0.0)
        : m_cost { cost }
    {
    }
};
```

Inheritance

- Order of construction of derived class: call of specific constructor
  - If not explicitly specified, default constructor of base class is called automatically in the constructor of derived class
  - Do not try to initialize base class members from derived constructor, but call the base class custom constructor instead
- The order of destruction will be in **reverse** of constructors calls

```
class Base
{
public:
    int m_id {};

    Base(int id=0)
        : m_id { id }
    {
    }
};

class Derived: public Base
{
public:
    double m_cost {};

    Derived(double cost=0.0)
        : Base (1), m_cost { cost }
    {
    }
};
```

Inheritance

- Multiple inheritance is allowed in C++
  - Means that the derived class inherits from all base classes

```
class USBDevice
{
[...]
public:
    long getID() const { return m_id; }
};

class NetworkDevice
{
[...]
public:
    long getID() const { return m_id; }
};

class WirelessAdapter: public USBDevice, public NetworkDevice
{
};

int main()
{
    WirelessAdapter ad;
    std::cout << ad.USBDevice::getID();

    return 0;
}
```

In case of ambiguities, the origin base class should be specified

ACTIA®

Polymorphism

- Polymorphism is the ability of an object or function to take multiple forms
- Generally, it allows accessing object through the same interface
- We call interface a pure virtual class (which have at least one pure virtual method)

```
class Interface
{
public:
     virtual void pureVirtual() = 0;
};
```

- Each derived class (subclass) can have its own behavior for given method by implementing its own version of that method

Polymorphism

- Polymorphism types in C++
  - Compile time polymorphism (will be detailed in next chapter)
    - Functions overloading
    - Operators overloading
  - Runtime polymorphism (called also early binding polymorphism)
    - Compiler cannot decide which form of object to be used, it will be decided at runtime

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

ACTIA®

Polymorphism

- Runtime polymorphism conditions in C++ (mandatory)
  - Use pointer or reference on the base class
  - Functions must be virtual (keyword used to indicate that the non-static function support the dynamic dispatch)

Pointer or reference used

virtual keyword in base class:

```
class Base
{
public:
     virtual void virtualFunc() { std::cout << "Base class impl" << std::endl; }
};
class Derived : public Base
{
public:
     void virtualFunc() { std::cout << "Derived class impl" << std::endl; }
};
```

virtual keyword optional in derived class:

```
int main()
{
     Derived derived;
     Base * pdrv = new Derived();
     Base& drv = derived;
     pdrv->virtualFunc();
     drv.virtualFunc();
     delete drv;

     return 0;
}
```

Polymorphism

- Base class passed in function parameter

Function should be virtual

```
class Base
{
public:
     virtual void virtualFunc() const { std::cout << "Base class impl" << std::endl; }
};
class Derived1 : public Base
{
public:
     void virtualFunc() const { std::cout << "Derived1 class impl" << std::endl; }
};
class Derived2 : public Base
{
public:
     void virtualFunc() const { std::cout << "Derived2 class impl" << std::endl; }
};
```

Should be passed by reference or by address

```
void callVirtual(const Base& base)
{
     base.virtualFunc();
}
int main()
{
     Derived1 drv1;
     Derived2 drv2;
     // displays Derived1 class impl
     callVirtual(drv1);
     // displays Derived2 class impl
     callVirtual(drv2);
     return 0;
}
```

Polymorphism

- Heterogeneous lists: collection of objects based on same interface

Derived classes should be polymorphic

```
Derived1 drv11;
Derived1 drv12;
Derived2 drv21;
Derived2 drv22;
std::vector<Base* > vect2 {&drv11, &drv21, &drv12, &drv22};
for(int i = 0; i < vect2.size(); i++)
{
     vect2[i]->virtualFunc();
}
```



Do not call virtual method in constructor of destructor

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

Overloading and overriding

- Function Overloading: have the same function with same name in the same scope
- Must have different function signature
  - Function signature doesn't include return type)
  - Number and type of parameters are use to change function signature

```
int foo(int x, int y);
void foo(int x, int y);
```

same signature => compilation error

```
void virtualFunc() const;
void virtualFunc();
```

different signature => OK

- Compiler may perform implicit conversion for overloading matc

```
void print(int);
void print(double);
```

Implicitly converted to int and print(int) is called → `print(true);`

Overloading and overriding

- Functions overriding: redefining a function in derived class
  - Same function name and parameters
  - Base function must be declared as virtual
- Add «override» key optionally to indicate override in derived class (generate compile error in case of signature mismatch)

```
class Base
{
public:
     virtual void virtualFunc() const { std::cout << "Base class impl" << std::endl; }
};
class Derived1 : public Base
{
public:
     void virtualFunc() const override { std::cout << "Derived1 class impl" << std::endl; }
};
```

Overloading and overriding

- Pure virtual function or abstract function
  - Function that doesn't have a body
  - Cannot be called and must be overriden by derived classes
  - Add «= 0» at the end of pure virtual function declaration

```
void pureVirtualConst() const = 0;
void pureVirtual() = 0;
```

  - Generally used to give a behavior that is undefined in base class but must be defined only by derived classes

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

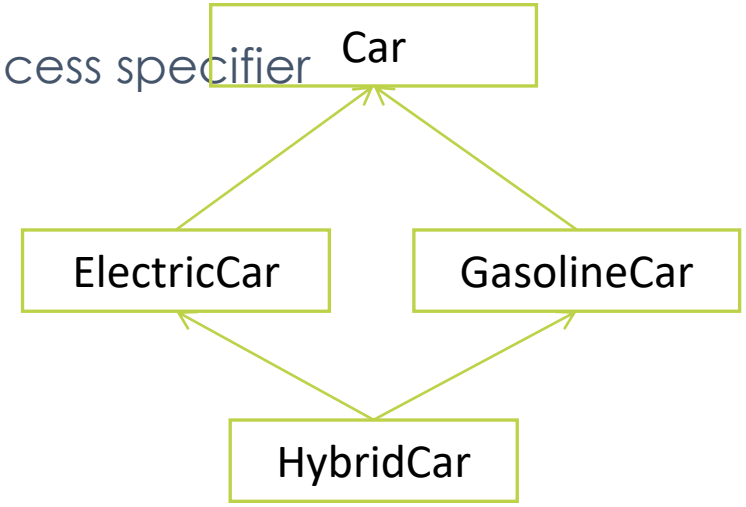
ACTIA®

Overloading and overriding

- Pure virtual class or abstract class or interface is a class that have at least one pure virtual function
  - An abstract class can have definition of other non-pure virtual function than pure virtual function
  - An abstract class cannot be instantiated (can just be inherited)

Overloading and overriding

- Virtual base class: diamond problem illustration
  - When instatiating the lower derived class, the upper base class will be instantiated twice
  - Solution: add virtual key word to the inheritance access specifier
  - This allows sharing the base class
  - The lower derived class will be then responsible for base class




```
class Car
class ElectricCar : virtual public Car
class GasolineCar : virtual public Car
class HybridCar : public ElectricCar, public GasolineCar
```

Encapsulation and abstraction

- Encapsulation: OOP concept that allows hiding details about object implementation from user
  - User then accesses the object using public interfaces that manages the how to access data and that protects the object implementation
  - Leads to have all member variables private (or protected for inheritance)
- Access functions:
  - Getters: used to get the value of private member varic
  - Setters: used to set the value of private member variables

```
type_name getVariable() const;
type_name getVariable();
void setVariable(const type_name& value);
```

Encapsulation and abstraction

- Abstraction: OOP concept that aims to hide objects data from outside world and provide only the necessary information.
  - Using separate header and source files
  - Use access specifiers to put public only the information that should be presented to user

Return to inheritance

- «final» specifier: used in Functions to prohibit overrding it

```
class A
{
public:
     virtual std::string getName() { return "A"; }
};
class B : public A
{
public:
     std::string getName() override final { return "B"; } // okay
};
class C : public B
{
public:
     std::string getName() override { return "C"; } // compile error: overrides B::getName(), which is final
};
```

Return to inheritance

- «final» specifier: used in classes to prohibit inheriting from it

```
class A
{
public:
     virtual std::string getName() { return "A"; }
};
class B final : public A // note use of final specifier here
{
public:
     std::string getName() override { return "B"; }
};
class C : public B // compile error: cannot inherit from final class
{
public:
     std::string getName() override { return "C"; }
};
```

Operators overloading

- Introduction to operators: In C++, operators are considered as functions and can be overloaded
- When we perform operation «x + y», compiler uses built-in version of '+' operator

  it will be translated to **operator+**(x, y)
- If x and y are built-in types, no problem is faced since built-in operators are already defined

```
class A; // suppose that A is already defined
A a1;
A a2;
A res = a1 + a2; // try to apply operator + to a1 and a2
```

ined, compiler tries to perform casting to exiting

→ Compile error

- Otherwise, compile error is generated

Operators overloading

- Copy assignment operator:class_name& **operator=**(const class_name&)
- Must be overloaded as member function of class
- As special operator, unlike other operators, when neither provided nor deleted, compiler provides default one

```
class Str
{
public:
     Str(const std::string& value): m_value{value} {}
     Str& operator=(const Str& str) {
          m_value = str.m_value;
          return *this;
     }
     Str(const Str& str): m_value{str.m_value}
     {
     }
private:
     std::string m_value;
};
```

```
int main()
{
     Str str("Test ");
     Str val = str; // call of copy constructor
     val = str; // call of assignment operator
     return 0;
}
```

Operators overloading

- Copy assignment operator:
  - If class is simple, we can lean on default version of copy constructor or assignment operator
  - If class contains tricky things (like pointers), we have to define them: this is called deep copy

```
private:
    void deepCopy(const A& a) {
        if (m_ptr) delete m_ptr;
        if (a.m_ptr) {
            m_ptr = new int;
            *m_ptr = *a.m_ptr;
        }
    }
    int * m_ptr;
```

y of all memb

v pointer (sin

```
public:
    A(): m_ptr{nullptr} {}
    A(const A& a) {
        deepCopy(a);
    }
    A& operator=(const A& a) {
        if (this != &a) { deepCopy(a) }
        return *this;
    }
```

ıt object in copy operation)

Operators overloading

- Move assignment operator: class_name& **operator=**(class_name&&)
- Must be overloaded as member function of class
- As special operator, unlike other operators, when neither provided nor deleted, compiler provide

```
class A {
public:
    A(): m_ptr{nullptr} {}
    A(A&& a) : m_ptr{a.m_ptr} {
        a.m_ptr = nullptr;
    }
    A& operator=(A&& a) {
        if (this != &a) {
            m_ptr = a.m_ptr;
            a.m_ptr = nullptr;
        }
        return *this;
    }
private:
    int * m_ptr;
};
```

```
int main()
{
    A a;
    a = A(); // call of move assignment operator
    A a1 = std::move(A()); // call move constructor
    A a2 = std::move(a1); // call move constructor
    return 0;
}
```

Operators overloading

- Move assignment operator:
  - If class is simple, we can lean on default version of copy constructor or assignment operator
  - It will be better to define move constructor or move assignment operator when move is allowed for object in question
- Move operation should leave the moved object in valid state. Since moved object is a temporary object (r-value), it will be deleted anyway and if we have pointers they will be deleted (and memory that it is pointing to will be lost)

☐ Provisoire ☐ Public ☐ Interne ☒ Confidentiel ☐ Secret

ACTIA®

Operators overloading

- To overload an operator with specific operands, operator keyword is used followed by the operator
- List of possible operators contains all built-in operator list
- Good practice
  - Operators that do not modify their operands (e.g. arithmetic operators) should generally return results by value.
  - Operators that modify their leftmost operand (e.g. pre-increment, any of the assignment operators) should generally return the leftmost operand by reference.

☐ Provisoire ☐ Public ☐ Interne ☒ Confidentiel ☐ Secret

ACTIA®

Operators overloading

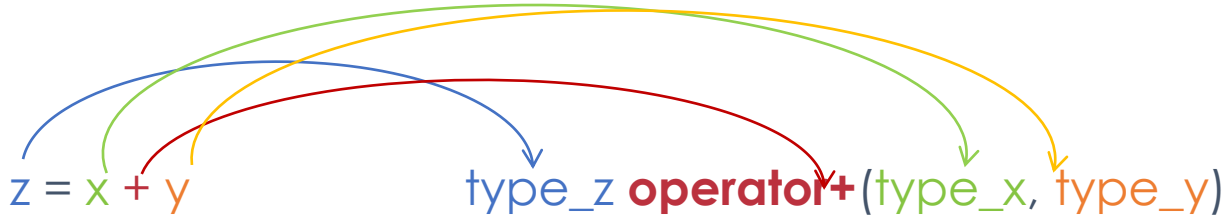
- Three ways to implement operator overloading, as:
  - Friend function
  - Member function
  - Normal function (non-member function)
- As reminder we have 4 types of operators, only 2 are applicable for overloading
  - Unary
  - Binary

☐ Provisoire ☐ Public ☐ Interne ☒ Confidentiel ☐ Secret

ACTIA®

Operators overloading

- Overloading principle



- Return type can be a value or a reference, generally operators that doesn't modify the object like arithmetic operators (+, *, /, ...) are returned by value and operators that modify the object like stream '<<' operator are returned by reference

Operators overloading

- Overloading using friend function: operator definition function will be declared as friend to the class and can be defined inside or outside the class

```
class A
{
public:
     A(const int& value): m_value{value} {}
     int getA() const { return m_value; }
     A& operator=(A&& a) { m_value = a.m_value; return *this; }
     A(A&& a) : m_value {a.m_value} {}
     friend A operator+(const A& a, int);
     friend A operator+(int x, const A& a)
     {
          A ares(x + a.m_value);
          return ares;
     }
private:
     int m_value;
};

A operator+(const A& a, int x)
{
     A ares(x + a.m_value);
     return ares;
}
```

```
int main()
{
     A a(1);
     a = a + 2;
     std ::cout << "Result " << a.getA();
     a = 3 + a;
     std ::cout << "Result " << a.getA();
     return 0;
}
```

Operators overloading

- Overloading using friend function: IO stream operator «<<»
  - Used to display class information by just putting object instance after the operator in

```
class Frame
{
public:
     enum class Origin
     {
          Server,
          Client
     };
     Frame(int length, const std::string& payload, Origin origin):
          m_length{length}, m_payload{payload}, m_origin{origin} {}
     int length() const { return m_length; }
     std::string payload() const { return m_payload; }
     Origin origin() const { return m_origin; }
private:
     int m_length;
     std::string m_payload;
     Origin m_origin;
};
```

Hard to read and requires getters to be defined

```
int main()
{
     Frame f(10, "Test frame", Frame::Origin::Client);
     std::cout << "Frame details: length " << f.length()
               << " data " << f.payload() <<  " origin "
               << (f.origin() == Frame::Origin::Server ?
                                   "Server" : "Client")
               << std::endl;
     return 0;
}
```

Operators overloading

- Overloading using friend function: IO stream operator «<<»
  - Example of use of output stream operator overloading

```
class Frame
{
public:
     enum class Origin
     {
          Server,
          Client
     };
     Frame(int length, const std::string& payload, Origin origin):
          m_length{length}, m_payload{payload}, m_origin{origin} {}

     friend std::ostream& operator<<(std::ostream& out, const Frame& frame);
private:
     int m_length;
     std::string m_payload;
     Origin m_origin;
};
```

```
std::ostream& operator<<(std::ostream& out, const Frame& frame)
{
     out << "length " << frame.m_length
               << " data " << frame.m_payload <<  " origin "
               << (frame.m_origin == Frame::Origin::Server ?
                                     "Server" : "Client");
     return out;
}

int main()
{
     Frame f(10, "Test frame", Frame::Origin::Client);
     std::cout << "Frame details: " << f << std::endl;
     return 0;
}
```

Operators overloading

- Overloading using normal function: operator definition will be in a simple function

```
class A
{
public:
     A(const int& value): m_value{value} {}
     int getA() const { return m_value; }
     A& operator=(A&& a) { m_value = a.m_value; return *this; }
     A(A&& a) : m_value {a.m_value} {}
private:
     int m_value;
};

A operator+(const A& a, const A& b)
{
     int x = a.getA() + b.getA();
     return A(x);
}
```

```
int main()
{
     A a(1);
     A a1(2);
     std::cout << (a + a1).getA() << std::endl;
     return 0;
}
```

Operators overloading

- Overloading using member function:
  - Not everything can be overloaded as member function because the overloaded operator must be added as a member of the left operand.

```
class A
{
public:
     explicit A(const int& value): m_value{value} {}
     int getA() const { return m_value; }
     A& operator=(A&& a) { m_value = a.m_value; return *this; }
     A(A&& a) : m_value {a.m_value} {}

     // member operator
     A operator+(std::string& str);
private:
     int m_value;
};
```

```
A A::operator+(std::string& str)
{
     m_value += std::atoi(str.c_str());
     return A(m_value);
}

int main()
{
      A a(2);
      std::string str("23");
      a = a + str;
      std ::cout << "Result sum string " << a.getA() << std::endl;
      return 0;
}
```

Operators overloading

- Overloading exercie
  - Implement a class named Digit that contains an int attribute and overload the increment and decrement operators

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

ACTIA®

Operators overloading

- Overloading operators examples: see examples source code for operators
  - Unary: negation '-'
  - Binary:
    - Comparison: '==', '>=', '<='
    - Subscript: '[]'
    - Parantheses (functor) '()' with arbitrary parameter number
    - Typecast samples

☐ Provisoire ☐ Public ☐ Interne ☒ Confidentiel ☐ Secret

ACTIA®

Objects relationships

- To have a well structured program, it must be composed by many objects and these objects have to interact between each others
- Relationships can be described by words (even before programming during the conception phase)
  - Car «is-a» vehicle
  - Car «has-a» door and «has-a» list of wheels
  - Programmer «uses-a» keyboard
  - Computer «Depends-on» electricity
  - Employee «member-of» employees list

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

Objects relationships

- **Composition**: «has-a» relationship: object is composed by smaller objects. Create simpler classes to simplify complex classes
  - The smaller object must be part of the global object
  - The smaller object belongs to one object at a time
  - The smaller object existence is managed by global object (it won't exist if gloabal object no longer exists)
  - The smaller object doesn't know about the global object existen

```
class B {
};
class C {
};
class A {
private:
     B m_b;
     C * m_ptr;
};
```

Objects relationships

- **Aggregation**: «has-a» relationship as well: it is similar to composition, but
  - The smaller object must be part of the global object
  - The smaller object **can** belong to **many** objects at a time
  - The smaller object existence is **not** managed by global object: its existence doesn't depend on global object existence
  - The smaller object doesn't know about the global object existence

Objects relationships

- **Aggregation**:
  - When global object is destructed, it is not responsible for aggregated objects destruction

```
class Employee {
};

class Company {
public:
     Company(const Employee& employee) : m_employee{employee}
     {}
private:
     const Employee& m_employee;
};

int main()
{
     Employee employee;
     {
          Company company(employee);
     }
     // company is distructred, but employee still alive
     return 0;
}
```

Objects relationships

- **Association**: «Uses-a» relationship
  - The associated object is not related to the object
  - The associated object can belong to more than one object
  - The associated object existence is not managed by the object
  - The associated object may or may not know about object existence
- Unlike composition and aggregation, association may be bidirectional (object are aware of each other)

Objects relationships

Association: implementation

```
class Patient;

class Doctor
{
public:
     Doctor(const std::string& name) : m_name{ name } {
     }
     void addPatient(Patient& patient)  {
          // add to patients list
          // add dotor to patient
          patient.addDoctor(*this);
     }
};
class Patient
{
public:
     Patient(const std::string& name) : m_name{ name } {
     }
     void addDoctor(const Doctor& doctor) {
          // add to doctors list
     }
};
```

```
int main()
{
     Patient patient{ "PName" };
     Doctor doctor{ "DName" };

     doctor.addPatient(patient);
     return 0;
}
```

Objects relationships

Summary:

Property	Composition	Aggregation	Association
Relationship type	Whole/part	Whole/part	Otherwise unrelated
Members can belong to multiple classes	No	Yes	Yes
Members' existence managed by class	Yes	No	No
Directionality	Unidirectional	Unidirectional	Unidirectional or bidirectional
Relationship verb	Part-of	Has-a	Uses-a

☐ Provisoire ☐ Public ☐ Interne ■ Confidentiel ☐ Secret

ACTIA®

Objects relationships

- **Dependency**: «Depends-on» relationship, occurs when one object invokes another object's functionality in order to accomplish some specific task.
- This is a weaker relationship than an association, but still, any change to object being depended upon may break functionality in the (dependent) caller.
- A dependency is always a unidirectional relationship.

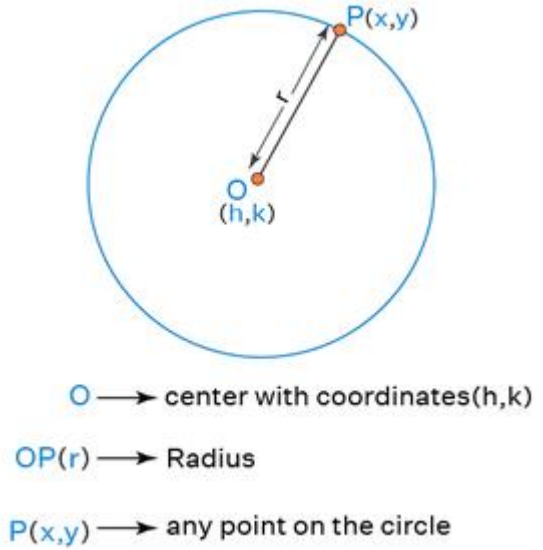
```
class Doctor
{
public:
     Doctor(const std::string& name) : m_name{ name } {
     }
     // Doctor has a dependency on std::ostream here
     friend std::ostream& operator<< (std::ostream& out, const Doctor& doctor)
     {
          out << "Doctor name=" << doctor.m_name;
          return out;
     }
};
```

Exercice

- Create a class Point that holds two integers which are x and y coordinates.
- Create a pure virtual class Shape that contains
  - Pure virtual print method that gets as input an ostream and returns it back
  - A pure virtual method that checks if a Point is in the shape outline
  - Overload of the stream operator
- Create a class circle that inherits from Shape and has an attribute ‘center’ Point and radius value
  - Perform necessary actions, knowing that the Circle is no longer inherited
  - Overload the operator ‘+=’ to be able to add Point(s) to the circle outline

Exercice

- Write Main function to highlight the implemented code
- (x - h)2 + (y - k)2 = r2 where (h,k) are the center point coordinated, r is the radius and (x, y) are corrdinated of a point in the circle outline



Quizzzzz

- C++

ACTIA®
ENGINEERING SERVICES

AN ENTREPRENEURIAL SPIRIT

TO MOVE FORWARD TOGETHER

ACTIA Engineering Services

☐ Provisoire ☐ Public ☐ Interne ☒ Confidentiel ☐ Secret

ACTIA®